Universidad El Bosque · Ingeniería de Sistemas

Bases de Datos 2 · Docente: Christian Felipe Duarte

Taller MQL en MongoDB

Inserción, consulta, actualización y agregación sobre el dataset sample_airbnb

Autor
David Felipe Bejarano Montes

Base y colección
sample_airbnb · listingsAndReviews

Fecha
3 de noviembre de 2026

Contenido

Resumen
1. Inserción de documentos
2. Consultas con find
3. Actualización de documentos
4. Pipelines de agregación

Resumen

El taller pide resolver 30 ejercicios de MQL (MongoDB Query Language) sobre la colección `listingsAndReviews` del dataset de ejemplo de Airbnb que trae MongoDB Atlas. Los ejercicios van de menos a más: insertar documentos, consultarlos con `find`, modificarlos y, al final, analizarlos con pipelines de agregación.

Esta página reúne el enunciado resumido, el script de cada ejercicio y una nota con lo que hay que tener en cuenta al ejecutarlo (tipos de dato, comportamiento de los operadores, orden de ejecución). El script completo se puede descargar y abrir como *playground* en MongoDB for VS Code.

> **Sobre la ejecución**
>
> Los scripts están escritos con la estructura real del dataset `sample_airbnb`, pero esta versión no incluye capturas ni resultados de ejecución. Los ejercicios 1 a 5 y 16 a 20 modifican la colección, y el 17 depende del 2, así que conviene ejecutarlos en orden.

Bloque	Ejercicios	Operaciones
Inserción	1 a 5	`insertOne`, `insertMany`, unicidad de `_id`
Consultas	6 a 15	`find`, proyección, `$gt`, `$lt`, `$and`, `$in`, regex, arrays, `sort`, `limit`
Actualización	16 a 20	`updateOne`, `updateMany`, `$set`, `$push`, `$inc`, `$pull`, `upsert`
Agregación	21 a 30	`$match`, `$group`, `$unwind`, `$project`, `$bucket`, `$lookup`, `$year`

1. Inserción de documentos

Ejercicio 1. insertOne básico

Insertar un listing con `name`, `summary`, `property_type: "Cabin"` y `price: 150.00`.

Ejercicio 1 · insertOne básico

```
col.insertOne({
  name: "Cozy Cabin Retreat",
  summary: "Cabana acogedora rodeada de bosque, ideal para desconectarse.",
  property_type: "Cabin",
  price: NumberDecimal("150.00")
});
```

> Para tener en cuenta
>
> El `price` se guarda como `NumberDecimal` porque en `sample_airbnb` el precio es de tipo Decimal128. Si se insertara un número normal, la colección quedaría con dos tipos distintos en el mismo campo. Como no se envía `_id`, MongoDB genera un `ObjectId`.

Ejercicio 2. insertOne con objeto anidado (address)

Insertar un listing con el objeto anidado `address`, con `country: "Chile"` y `market: "Santiago"`.

Ejercicio 2 · insertOne con objeto anidado (address)

```
col.insertOne({
  name: "Casa Patagonia Lodge",
  summary: "Casa de montana con vista al lago.",
  property_type: "House",
  price: NumberDecimal("120.00"),
  address: {
    country: "Chile",
    market: "Santiago"
  }
});
```

> Para tener en cuenta
>
> `address` se escribe como un documento dentro del documento. Más adelante se accede a sus campos con notación de punto (`"address.country"`).

Ejercicio 3. insertMany simple

Insertar en una sola operación “Studio de Arte Urbano” (80.00) y “Loft Minimalista” (110.00).

Ejercicio 3 · insertMany simple

```
col.insertMany([
  { name: "Studio de Arte Urbano", price: NumberDecimal("80.00") },
  { name: "Loft Minimalista", price: NumberDecimal("110.00") }
]);
```

> Para tener en cuenta
>
> `insertMany` es ordenado por defecto: si un documento falla, los siguientes no se insertan. Con `{ ordered: false }` el motor intenta insertar todos.

Ejercicio 4. insertMany con arrays (amenities)

Insertar listings que incluyan un array `amenities` con al menos tres elementos.

Ejercicio 4 · insertMany con arrays (amenities)

```
col.insertMany([
  {
    name: "Apartamento Centro Historico",
    property_type: "Apartment",
    price: NumberDecimal("95.00"),
    amenities: ["Wifi", "Kitchen", "Free parking"]
  },
  {
    name: "Casa de Campo Los Alamos",
    property_type: "House",
    price: NumberDecimal("130.00"),
    amenities: ["Wifi", "Heating", "Kitchen", "Free parking"]
  }
]);
```

> Para tener en cuenta
>
> Los arrays no necesitan declararse antes: basta con escribir la lista. El enunciado habla de un listing; inserté dos para dejar un dato con “Heating”, útil para probar el ejercicio 12.

Ejercicio 5. Validación de _id duplicado

Insertar un listing con un `_id` que ya existe, observar el error y describirlo.

Ejercicio 5 · Validación de _id duplicado

```
// Se toma el _id de un documento que ya existe y se intenta reutilizar.
const idExistente = col.findOne({}, { _id: 1 })._id;
try {
  col.insertOne({ _id: idExistente, name: "Listing con _id repetido", price:
NumberDecimal("50.00") });
} catch (e) {
  print("Error esperado: " + e.message);   // E11000 duplicate key error
}
```

> Para tener en cuenta
>
> Resultado esperado: error `E11000 duplicate key error collection: sample_airbnb.listingsAndReviews index: _id_` . El índice sobre `_id` es único y se crea de forma automática, así que MongoDB rechaza el documento. En vez de escribir un `_id` a mano, el script lee uno que ya existe para que el ejercicio funcione igual sin importar el dataset.

2. Consultas con find

Ejercicio 6. Listings de Brazil (name y price)

Mostrar `name` y `price` de los listings ubicados en Brazil.

Ejercicio 6 · Listings de Brazil (name y price)

```
col.find(
  { "address.country": "Brazil" },
  { _id: 0, name: 1, price: 1 }
);
```

> Para tener en cuenta
>
> El segundo argumento de `find` es la proyección. `_id: 0` oculta el identificador, que aparece por defecto.

Ejercicio 7. Más de 5 camas ($gt)

Listings con más de 5 `beds`, mostrando `name` y `number_of_reviews`.

Ejercicio 7 · Más de 5 camas ($gt)

```
col.find(
  { beds: { $gt: 5 } },
  { _id: 0, name: 1, number_of_reviews: 1 }
);
```

> Para tener en cuenta
>
> `$gt` es “mayor que” estricto. Los documentos sin el campo `beds` no cumplen la condición.

Ejercicio 8. Precio menor a 75 ($lt)

Listings con `price` menor a 75, proyectando `name`, `price` y `property_type`.

Ejercicio 8 · Precio menor a 75 ($lt)

```
col.find(
  { price: { $lt: 75 } },
  { _id: 0, name: 1, price: 1, property_type: 1 }
);
```

> Para tener en cuenta
>
> El precio está guardado como Decimal128; MongoDB compara tipos numéricos distintos por su valor, así que se puede filtrar con el número 75 sin convertirlo.

Ejercicio 9. $and con objeto anidado: Apartment en New York

Listings de tipo “Apartment” ubicados en el `market` “New York”.

Ejercicio 9 · $and con objeto anidado: Apartment en New York

```
col.find({
  $and: [
    { property_type: "Apartment" },
    { "address.market": "New York" }
  ]
});
```

> Para tener en cuenta
>
> Con `$and` ambas condiciones deben cumplirse. Aquí también habría bastado el filtro implícito `{ property_type: "Apartment", "address.market": "New York" }`. La ciudad está en `address.market`, no en un campo `city`.

Ejercicio 10. $in: House o Condominium

Listings cuyo `property_type` sea “House” o “Condominium”.

Ejercicio 10 · $in: House o Condominium

```
col.find({ property_type: { $in: ["House", "Condominium"] } });
```

> Para tener en cuenta
>
> `$in` equivale a varios `$or` sobre el mismo campo.

Ejercicio 11. Regex sin distinción de mayúsculas: "Luxury"

Listings cuyo `name` contenga “Luxury”, sin distinguir mayúsculas.

Ejercicio 11 · Regex sin distinción de mayúsculas: "Luxury"

```
col.find({ name: { $regex: /Luxury/i } });
```

> Para tener en cuenta
>
> La opción `i` de la expresión regular ignora mayúsculas. Un regex sin ancla no usa bien un índice, así que sobre colecciones grandes es lento.

Ejercicio 12. Búsqueda en array: amenity "Heating"

Listings que ofrezcan “Heating” entre sus amenities.

Ejercicio 12 · Búsqueda en array: amenity "Heating"

```
col.find({ amenities: "Heating" });
```

> Para tener en cuenta
>
> Cuando el campo es un array, `{ amenities: "Heating" }` coincide si algún elemento es igual al valor.

Ejercicio 13. $size: exactamente 20 amenities

Listings con exactamente 20 amenities.

Ejercicio 13 · $size: exactamente 20 amenities

```
col.find({ amenities: { $size: 20 } });
```

> Para tener en cuenta
>
> `$size` solo admite un número exacto; no sirve para rangos como "más de 10" (por eso el ejercicio 21 usa `$expr`).

Ejercicio 14. Los 10 más caros (sort + limit)

Los 10 listings más caros, con `name`, `price` y `address.country`.

Ejercicio 14 · Los 10 más caros (sort + limit)

```
col.find(
  {},
  { _id: 0, name: 1, price: 1, "address.country": 1 }
).sort({ price: -1 }).limit(10);
```

> Para tener en cuenta
>
> En el ordenamiento, `-1` es descendente. `sort` y `limit` se aplican después del filtro y antes de la proyección en el resultado final.

Ejercicio 15. Sin last_review (ausente o nulo), por number_of_reviews desc

Listings sin `last_review` (ausente o nulo), ordenados por `number_of_reviews` descendente.

Ejercicio 15 · Sin last_review (ausente o nulo), por number_of_reviews desc

```
// { last_review: null } coincide con campo nulo y con campo ausente.
col.find(
  { last_review: null },
  { _id: 0, name: 1, summary: 1, description: 1 }
).sort({ number_of_reviews: -1 });
```

> Para tener en cuenta
>
> El filtro `{ last_review: null }` coincide con los documentos donde el campo es nulo y con los que no lo tienen. Si se quisiera distinguir, se usaría `$exists`.

3. Actualización de documentos

Ejercicio 16. updateOne simple

Cambiar el `property_type` de “Cozy Cabin Retreat” a “Entire home/apt”.

Ejercicio 16 · updateOne simple

```
col.updateOne(
  { name: "Cozy Cabin Retreat" },
  { $set: { property_type: "Entire home/apt" } }
);
```

> Para tener en cuenta
>
> `updateOne` modifica solo el primer documento que cumpla el filtro. Si hubiera dos listings con el mismo nombre, cambiaría uno solo.

Ejercicio 17. updateOne con $push sobre el listing del ejercicio 2

Añadir la amenity “Smart TV” al listing insertado en el ejercicio 2.

Ejercicio 17 · updateOne con $push sobre el listing del ejercicio 2

```
// El documento del ejercicio 2 no tenia amenities: $push crea el array.
col.updateOne(
  { name: "Casa Patagonia Lodge" },
  { $push: { amenities: "Smart TV" } }
);
```

> Para tener en cuenta
>
> `$push` agrega un elemento al final del array; si el campo no existe, lo crea. Si se quisiera evitar duplicados se usaría `$addToSet`. Depende de que el ejercicio 2 se haya ejecutado antes.

Ejercicio 18. updateMany con $inc: Spain

Incrementar en 1 el `number_of_reviews` de todos los listings de Spain.

`Ejercicio 18 · updateMany con $inc: Spain`

```
col.updateMany(
  { "address.country": "Spain" },
  { $inc: { number_of_reviews: 1 } }
);
```

> Para tener en cuenta
>
> `updateMany` aplica el cambio a todos los documentos que coincidan. Si el campo no existe, `$inc` lo crea con el valor del incremento.

Ejercicio 19. updateMany con $pull: quitar "Cable TV"

Eliminar la amenity “Cable TV” de todos los listings que la contengan.

`Ejercicio 19 · updateMany con $pull: quitar "Cable TV"`

```
col.updateMany(
  { amenities: "Cable TV" },
  { $pull: { amenities: "Cable TV" } }
);
```

> Para tener en cuenta
>
> `$pull` elimina todos los elementos que coincidan con la condición. Es una modificación masiva sobre el dataset, así que conviene ejecutarla una sola vez.

Ejercicio 20. Upsert

Actualizar “Mansión de Prueba” y, si no existe, insertarla con precio 1500.00 y país United States.

`Ejercicio 20 · Upsert`

```
// El filtro aporta el name; $setOnInsert solo aplica si el documento se crea.
col.updateOne(
  { name: "Mansion de Prueba" },
  { $setOnInsert: { price: NumberDecimal("1500.00"), "address.country": "United States" } },
```

```
  { upsert: true }
);
```

> Para tener en cuenta
>
> Con `upsert: true`, si el filtro no encuentra nada, MongoDB crea un documento con los campos del filtro más los de la actualización. `$setOnInsert` hace que precio y país solo se escriban cuando el documento se crea, sin pisar uno existente.

4. Pipelines de agregación

Ejercicio 21. $match y $count: más de 10 amenities

Contar los listings con más de 10 amenities.

Ejercicio 21 · $match y $count: más de 10 amenities

```
col.aggregate([
  { $match: { $expr: { $gt: [{ $size: { $ifNull: ["$amenities", []] } }, 10] } } },
  { $count: "listings_con_mas_de_10_amenities" }
]);
```

> Para tener en cuenta
>
> `$size` dentro de `$match` no admite comparaciones, así que se usa `$expr` con `$gt`. `$ifNull` evita un error en los documentos sin el campo `amenities`.

Ejercicio 22. Precio promedio por país

Precio promedio de los listings por `address.country`.

Ejercicio 22 · Precio promedio por país

```
col.aggregate([
  { $group: { _id: "$address.country", precio_promedio: { $avg: "$price" } } },
  { $sort: { precio_promedio: -1 } }
]);
```

> Para tener en cuenta
>
> Los listings sin `address.country` quedan agrupados bajo `_id: null`. Se ordena por promedio para leer el resultado más fácil.

Ejercicio 23. Top 5 amenities ($unwind + $group)

Las 5 amenities más frecuentes en todo el dataset.

Ejercicio 23 · Top 5 amenities ($unwind + $group)

```
col.aggregate([
  { $unwind: "$amenities" },
  { $group: { _id: "$amenities", frecuencia: { $sum: 1 } } },
  { $sort: { frecuencia: -1 } },
  { $limit: 5 }
]);
```

> Para tener en cuenta
>
> `$unwind` genera un documento por cada elemento del array. Los documentos sin amenities (o con el array vacío) desaparecen del resultado.

Ejercicio 24. Cantidad de listings por property_type

Cantidad de listings por `property_type`, de mayor a menor.

Ejercicio 24 · Cantidad de listings por property_type

```
col.aggregate([
  { $match: { property_type: { $exists: true } } },
  { $group: { _id: "$property_type", cantidad: { $sum: 1 } } },
  { $sort: { cantidad: -1 } }
]);
```

> Para tener en cuenta
>
> Se filtra primero por `$exists` para no contar los documentos sin `property_type` bajo un grupo nulo.

Ejercicio 25. $project con conversión de tipo ($toInt)

Mostrar `name`, `price` y un campo `price_entero` con el precio convertido a entero.

Ejercicio 25 · $project con conversión de tipo ($toInt)

```
col.aggregate([
  { $project: { _id: 0, name: 1, price: 1, price_entero: { $toInt: "$price" } } }
]);
```

> **Para tener en cuenta**
>
> `$toInt` convierte el valor a entero. Si lo que se busca es redondear y no truncar, se usaría `$round` antes de la conversión. Conviene revisar en el resultado qué hace con los centavos.

Ejercicio 26. Promedio de review_scores_rating con >= 10 reviews

Promedio de `review_scores_rating` para los listings con al menos 10 reviews.

Ejercicio 26 · Promedio de review_scores_rating con >= 10 reviews

```
col.aggregate([
  { $match: { number_of_reviews: { $gte: 10 } } },
  { $group: { _id: null, promedio_rating: { $avg: "$review_scores.review_scores_rating" } } }
]);
```

> **Para tener en cuenta**
>
> Con `_id: null` el `$group` reúne todos los documentos del filtro en un solo grupo. `$avg` ignora los documentos donde el valor no existe.

Ejercicio 27. $bucket por rangos de precio

Agrupar los listings por rangos de precio: 0-100, 101-300, 301-1000 y 1000+.

Ejercicio 27 · $bucket por rangos de precio

```
// Los limites inferiores son inclusivos y los superiores exclusivos.
// "default" recoge todo lo que cae fuera de los limites (precio >= 1001).
col.aggregate([
  { $match: { price: { $exists: true } } },
  {
    $bucket: {
      groupBy: "$price",
      boundaries: [0, 101, 301, 1001],
      default: "1000+",
      output: { cantidad: { $sum: 1 } }
    }
  }
]);
```

> **Para tener en cuenta**
>
> En `$bucket` el límite inferior es inclusivo y el superior exclusivo. Por eso las fronteras son 0, 101, 301 y 1001: un precio de 100.50 queda en el primer rango. Lo que cae fuera de las fronteras (1001 o más) va al grupo `default`. Cada rango se identifica por su límite inferior.

Ejercicio 28. $lookup (teórico): colección hipotética "hosts"

Describir (teórico) un pipeline con `$lookup` para unir listings con una colección `hosts`.

Ejercicio 28 · $lookup (teórico): colección hipotética "hosts"

```
col.aggregate([
  {
    $lookup: {
      from: "hosts",
      localField: "host.host_id",
      foreignField: "host_id",
      as: "anfitrion"
    }
  },
  { $unwind: { path: "$anfitrion", preserveNullAndEmptyArrays: true } },
  { $project: { _id: 0, name: 1, "anfitrion.host_name": 1, "anfitrion.host_is_superhost": 1 } }
]);
```

> Para tener en cuenta
>
> Este ejercicio es teórico: en `sample_airbnb` los datos del anfitrión están incrustados en `host`, no en otra colección. `$lookup` hace una unión externa por la izquierda; devuelve un array (`as`), que `$unwind` aplana. Con `preserveNullAndEmptyArrays` se conservan los listings sin anfitrión.

Ejercicio 29. Promedio de accommodates por property_type

Promedio de `accommodates` por `property_type`, de mayor a menor.

Ejercicio 29 · Promedio de accommodates por property_type

```
col.aggregate([
  { $group: { _id: "$property_type", promedio_huespedes: { $avg: "$accommodates" } } },
  { $sort: { promedio_huespedes: -1 } }
]);
```

> Para tener en cuenta
>
> Ejercicio parecido al 22, ahora con `accommodates` (capacidad máxima de huéspedes).

Ejercicio 30. Listings por año de su última review

Cantidad de listings por año de su última review.

Ejercicio 30 · Listings por año de su última review

```
col.aggregate([
  { $match: { last_review: { $ne: null } } },
  { $group: { _id: { $year: "$last_review" }, listings: { $sum: 1 } } },
```

```
  { $sort: { _id: 1 } }
]);
```

> **Para tener en cuenta**
>
> El campo `last_review` es de tipo fecha, por eso funciona `$year`. Se excluyen los listings sin última review y el resultado se ordena por año.